Rapport Projet POO – Version 1 (POO-v1)

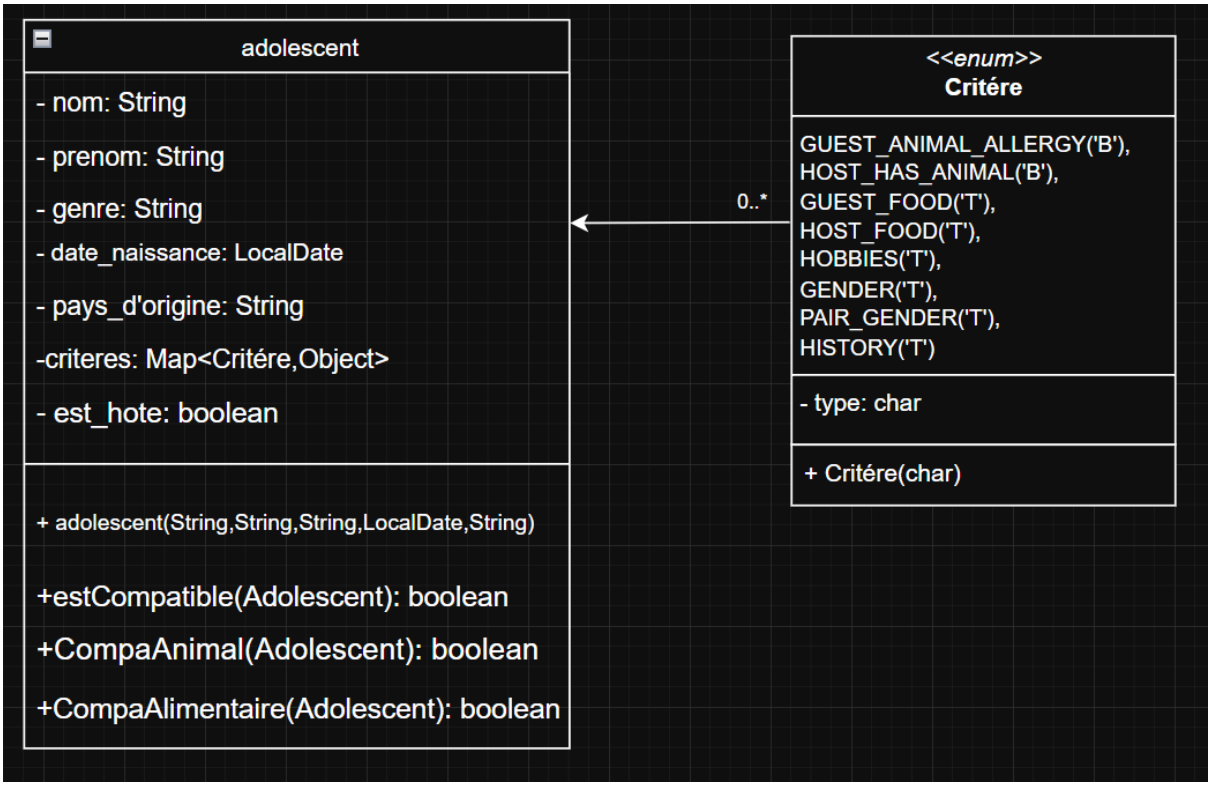
1. Fonctionnalités développées dans cette version

Dans cette première version du projet, nous avons mis en place la base de la modélisation des adolescents. Les éléments suivants ont été implémentés :

- La classe `Adolescent`, contenant les attributs principaux : nom, prénom, genre, date de naissance, pays d'origine, rôle (hôte ou invité), ainsi qu'une collection de critères.
- L'énumération `Critere`, qui définit différents types de critères : booleens (`'B'`) et textuels (`'T'`).
- La compatibilité entre deux adolescents a été gérée via deux critères :
    - Allergie aux animaux (`GUEST_ANIMAL_ALLERGY`) du visiteur et présence d'animaux chez l'hôte (`HOST_HAS_ANIMAL`)
    - Régime alimentaire (`GUEST_FOOD`) comparé avec (`HOST_FOOD`)

---

2. Schéma UML (POO-v1)



3. Scénarios de test

Les fonctionnalités ont été testées via une classe de test `TestAdolescent` qui instancie trois adolescents : un hôte (Enzo) et deux invités (Emile et Elyas).
 Les tests ont vérifié :

- L'ajout correct de critères textuels et booléens
- Le résultat de la méthode `estCompatible()` selon différentes valeurs des critères

Extrait :

```
//emile(guest)
Adolescent emile = new Adolescent(nom:"Camard", prenom:"Emile", genre:"M", LocalDate.of(year:2006, month:1, dayOfMonth:1), pays…"France", false);
//enzo(host)
Adolescent enzo = new Adolescent(nom:"Dewame", prenom:"Enzo", genre:"M", LocalDate.of(year:2006, month:4, dayOfMonth:1), pays…"France", true);
//elyas(guest)
Adolescent elyas = new Adolescent(nom:"Rabhiu", prenom:"Elyas", genre:"M", LocalDate.of(year:2005, month:4, dayOfMonth:1), pays…"France", false);


emile.ajouterValeur(Critere.GUEST_ANIMAL_ALLERGY, valeur:true);
emile.ajouterValeur(Critere.GUEST_FOOD, valeur:"vegetarian");
System.out.println("Émile est alergique ? " + emile.getCritere(Critere.GUEST_ANIMAL_ALLERGY));


enzo.ajouterValeur(Critere.HOST_HAS_ANIMAL, valeur:true);
enzo.ajouterValeur(Critere.HOST_FOOD, valeur:"vegetarian");
System.out.println("Enzo a un animal ? " + enzo.getCritere(Critere.HOST_HAS_ANIMAL));


elyas.ajouterValeur(Critere.GUEST_ANIMAL_ALLERGY, valeur:false);
elyas.ajouterValeur(Critere.GUEST_FOOD, valeur:"");
System.out.println("Élyas est alergique ? " + elyas.getCritere(Critere.GUEST_ANIMAL_ALLERGY));
System.out.println("quelle regime accepte enzo: " + enzo.getCritere(Critere.HOST_FOOD));
System.out.println();


System.out.println("compatibilité animal entre emile et enzo  : " + emile.CompaAnimal(enzo));
System.out.println("compatibilité nourriture entre emile et enzo: " + emile.CompaAlimentaire(enzo));

boolean compatibles = emile.estCompatible(enzo);
System.out.println("Émile est compatible avec Enzo ? " + compatibles);
System.out.println("Critères d'Enzo : " + enzo.getCriteres());
System.out.println("Critères d'Émile : " + emile.getCriteres());
System.out.println();



System.out.println("comptabilité animal entre elyas et enzo : " + elyas.CompaAnimal(enzo));
System.out.println("comptabilité alimentaire entre elyas et enzo: "+ elyas.CompaAlimentaire(enzo));
System.out.println("elyas et compatible avec enzo ? "+ elyas.estCompatible(enzo) );
```

Le résultat dans le terminal:

```
Émile est alergique ? true
Enzo a un animal ? true
Élyas est alergique ? false
quelle regime accepte enzo: vegetarian

compatibilité animal entre emile et enzo  : false
compatibilité animal entre emile et enzo  : false
compatibilité nourriture entre emile et enzo: true
Émile est compatible avec Enzo ? false
compatibilité nourriture entre emile et enzo: true
Émile est compatible avec Enzo ? false
Critères d'Enzo : {HOST_HAS_ANIMAL=true, HOST_FOOD=vegetarian}
Critères d'Enzo : {HOST_HAS_ANIMAL=true, HOST_FOOD=vegetarian}
Critères d'Émile : {GUEST_ANIMAL_ALLERGY=true, GUEST_FOOD=vegetarian}
Critères d'Émile : {GUEST_ANIMAL_ALLERGY=true, GUEST_FOOD=vegetarian}


comptabilité animal entre elyas et enzo : true
comptabilité animal entre elyas et enzo : true
comptabilité alimentaire entre elyas et enzo: true
elyas et compatible avec enzo ? true
```

4. Choix et évolution

Nous avons opté pour une `Map<Critere, Object>` afin de permettre de futurs ajouts des critères. Cela nous permettra d'ajouter facilement d'autres types de critères (hobbies, historique, genre, etc.) dans les versions futures.

---

5. Documentation

Une **JavaDoc** est en cours de rédaction pour toutes les méthodes liées aux fonctionnalités demandées (`estCompatible`, `ajouterValeur`, etc.).
Les méthodes du code sont commentées avec des commentaires standards pour nous aider à mieux comprendre notre code.
